

Neuronales Netzwerk – MNIST 3

Die nachfolgenden Aufgabenstellungen basieren auf den Ergebnissen von „Neuronales Netzwerk – MNIST 1“ und „Neuronales Netzwerk – MNIST 2“.

Die Funktionsweise und der Zweck der sog. *Confusion Matrix* sollten bekannt sein – zumindest in Bezug auf eine Binärklassifizierung. All das hat auch bei einem Mehrklassenproblem Gültigkeit. Jedoch umfasst die Matrix in diesem Fall mehrere Klassen. Bei der MNIST-Klassifizierung sind das 10, wie nachfolgender Abbildung 1 illustriert.

	pred	0	1	2	3	4	5	6	7	8	9
actual											
0	967	0	1	0	3	0	3	1	3	2	
1	0	1127	3	0	0	0	2	0	3	0	
2	2	1	1004	2	7	0	2	<u>5</u>	8	1	
3	0	0	2	991	1	1	0	4	4	7	
4	1	0	1	1	966	0	3	1	0	9	
5	2	1	0	19	2	850	4	1	9	4	
6	3	2	0	1	7	2	939	0	4	0	
7	1	5	<u>7</u>	2	2	0	0	1002	1	8	
8	2	0	5	5	5	1	1	4	948	3	
9	0	2	0	2	8	1	0	5	0	991	

Abbildung 1: Pandas MNIST-Confusion Matrix

Zu lesen ist das Teil wie folgt: Bspw. wurden sieben „7“ fälschlicherweise als „2“ klassifiziert (siehe rote Markierung). Im Vergleich dazu wurden fünf „2“ irrtümlicherweise als „7“ gelabelt (siehe grüne Markierung). Wie dem aufmerksamen Leser/der aufmerksamen Leserin sicherlich nicht entgangen ist, entspricht die Darstellung der *Confusion Matrix* nicht dem bekannten Output von `sklearn.metrics.confusion_matrix`. Der Autor dieser Zeilen hat sich erlaubt, mit `pandas.crosstab`¹ eine weitere Variante vorzustellen. Diese hat den Vorteil der Achsenbeschriftung (siehe „pred“ bzw. „actual“).

 18.3.1 Erstellen Sie die *Confusion Matrix* mit Pandas. Dass man die Daten wiederum aufbereiten muss, ist sicherlich keine Überraschung. Gegenwärtig liegen die relevanten Daten als 10-dim. Vektoren mit Wahrscheinlichkeiten vor. Die kann man so nicht übergeben. Die Implementierung der *Confusion Matrix* erwartet die Daten in dieser Form:

```
Y_pred = [2, 2, 1, ..., 4, 5, 6]
```

```
Y_actual = [7, 2, 1, ..., 4, 5, 6]
```

Die Numpy-Funktion `argmax`² „is your friend“.

¹ <https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.crosstab.html>

² <https://numpy.org/doc/stable/reference/generated/numpy.argmax.html>

Normalisierung der Daten

Gegenwertig bewegen Sie die Bildpunkte in einem Wertebereich zw. 0 und 255.

```
train_images[0]

array([[ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0],
       [ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0],
       [ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0],
       [ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0],
       [ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0],
       [ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0],
       [ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        18, 18, 18, 126, 136, 175, 26, 166, 255, 247, 127, 0, 0, 3,
        0,  0],
       ...])
```

Das ist für Neuronale Netze eher suboptimal. Das Lernen funktioniert in der Regel besser, wenn sich Datenpunkte z.B. zwischen 0 und 1 bewegen.

 18.3.2 Normalisieren Sie die Daten dahingehend, dass sich der Wertebereich der Bildpunkte zw. 0 und 1 bewegt. Trainieren Sie wiederum das Modell und vergleichen Sie die Ergebnisse.

Netzkomplexität

Als KI-Entwickler stellt man sich zwangsläufig irgendwann einmal die Frage „Wie komplex darf ein Netz überhaupt sein und wie lange sollte man trainieren?“. Leider gibt es keine allgemein gültige Antwort auf diese Frage. Im Grunde sollte man so lange trainieren, bis sich der *loss*-Wert nicht mehr viel ändert – also kleiner wird. Im Beispielsfall könnte man also statt 10 50 Epochen Zeit geben, um das Modell zu trainieren.

Es gilt: Das Netz passt sich an die Trainingsdaten an!

Nach 50 Epochen Training weist das Modell eine Genauigkeit in Bezug auf die Trainingsdaten von ca. 99.5% auf. Was aber möglicherweise der Überanpassung an die Trainingsdaten geschuldet ist. Denn bei den unbekannten Validierungsdaten liegt das Modell „nur“ mehr bei rund 98.32% richtig (vgl. „Neuronales Netzwerk – MNIST 2“).

```
model.evaluate(test_images, test_labels)

[0.1485150552543895, 0.9832000136375427]
```

- Testgenauigkeit nur minimal schlechter als die Trainingsgenauigkeit: In der Regel ist das der Überanpassung geschuldet. Um das festzustellen, kann man dem Modell

noch ein wenig mehr Komplexität hinzufügen (Stichwort Trail & Error). Eventuell sind da noch Zusammenhänge in den Daten, die unser Modell noch greifen könnte.

- Testgenauigkeit sehr viel schlechter als die Trainingsgenauigkeit: Das Modell ist viel zu komplex und hat viel zu spezifisch gelernt. Das Modell hat Besonderheiten gelernt, die ausschließlich in den Trainingsdaten gegeben sind.
- Beides schlecht: Man benötigt entweder mehr Daten oder die gewählte Architektur ist nicht entsprechend.

 18.3.3 Maximieren Sie die Testgenauigkeit. Dokumentieren Sie hierbei Ihre Ergebnisse.

Modell speichern/laden

Das Trainieren eines Modells kann einige Zeit in Anspruch nehmen. Damit man diesen Prozess im Fall des Falles nicht (ständig) wiederholen muss, ist es ratsam, das Modell zu speichern.

 18.3.4 Speichern und laden Sie das Modell mithilfe des Keras APIs³. Achten Sie darauf, dass die aktuelle *Sequential*-Instanz gelöscht wird.

³ https://keras.io/api/models/model_saving_apis/